

ARCHITECTURE BRIEF

Jazz Gen × Cycling '74 Max

A bidirectional WebSocket stream layer so the jam app and Max can talk in real time—beside the music-generation path, on one shared session clock.

DOCUMENT

Max WebSocket Stream Architecture

SCOPE

Construction, process layout, feature flow, Max use cases

DATE

July 22, 2026

STACK CONTEXT

Vite/React · Express + ws · Node for Max

Contents

1. Purpose & chosen model
2. What already exists in Jazz Gen
3. Process layout & execution
4. Wire protocol
5. Where the stream sits in the stack
6. How session features work through Max
7. Use cases Max can serve
8. Max-side construction
9. Latency & constraints
10. Build phases
11. Non-goals

1. Purpose & chosen model

This brief describes how to construct a **bidirectional MIDI and control stream** between Jazz Gen's Node backend and a Cycling '74 Max patch. It is not an audio-chunk pipeline and not a product roadmap. It is the transport and process layout that would sit *next to* today's procedural soloist (and a future ML inference client).

Chosen model — bidirectional peer. The Jazz Gen backend remains the authoritative form clock and session brain. Max connects as another WebSocket client: it can send live MIDI and control in, and receive soloist lines, backing parts, and clock/state out. Later, Max (or an ML service behind Max) can become the *producer* of soloist notes without rewriting the wire shape—only who generates `soloist_midi_out` changes.

What the stream carries

JSON events: beat-timed MIDI notes, session phase, BPM, chorus index, backing parts.

What it does not carry

PCM/WAV audio. Sound rendering stays in the browser SF2 engine and/or Max's own synths.

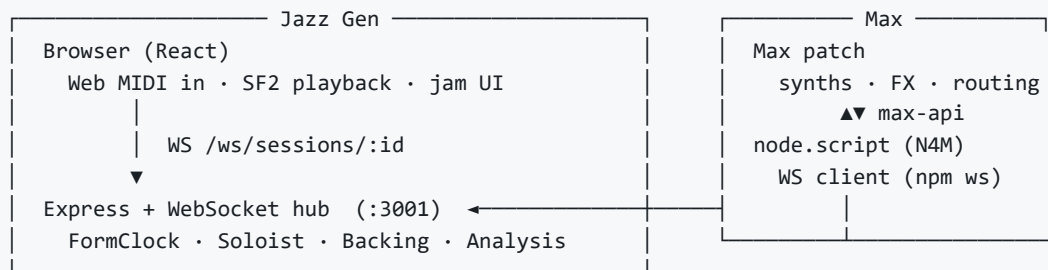
2. What already exists in Jazz Gen

Jazz Gen already runs a jam WebSocket. You are not inventing sockets from scratch— you are letting Max join the same session bus the browser already uses.

PIECE	LOCATION / BEHAVIOR
WS endpoint	<code>/ws/sessions/:id</code> on Express + <code>ws</code> (port 3001)
Connection registry	<code>sessionHandler.ts</code> — <code>Map<sessionId, Set<WebSocket>></code> with broadcast
Musical currency	<code>MidiNoteEvent</code> : <code>pitch</code> , <code>velocity</code> , <code>startBeat</code> , <code>durationBeats</code>
Form clock	<code>FormClockService</code> — authoritative beats, chorus flips, turn changes
“Model” today	Procedural TypeScript in <code>shared/music</code> via <code>SoloistService</code>
ML placeholder	Empty <code>MLInferenceClient</code> — stream layer sits beside this path

3. Process layout & execution

RUNTIME TOPOLOGY



PROCESS	ROLE
Backend (Node)	Session lifecycle, form clock, soloist generate, WS hub. One process hosts HTTP + WS.
Browser	Human UI, Web MIDI capture, SF2 playback. Unchanged as the primary surface.
Max	Patcher: instruments, effects, routing, live performance surface.
Node for Max	Separate Node process inside Max (<code>node.script</code>). Owns the <code>WebSocket</code> client, speaks JSON to the backend, bridges to Max via <code>max-api</code> .

Max has no first-class `WebSocket` object. The durable pattern is **Node for Max**: `node.script` + the `ws` npm package + `max-api` . No second “Max server” is required for v1—Max is a *client* of the existing hub, which keeps one clock and one session map.

Local jam startup order

- 1 Start the Jazz Gen backend on port 3001.
- 2 Open the browser app, create and start a session, note the `sessionId` .

- 3 In Max: run `script npm install` once, then `script start` on the bridge with that session id and `ws://localhost:3001/ws/sessions/<id>` .
- 4 N4M connects. The backend already fans out to every socket on that session, so Max receives the same broadcast as the browser (unless you later filter by role).

4. Wire protocol

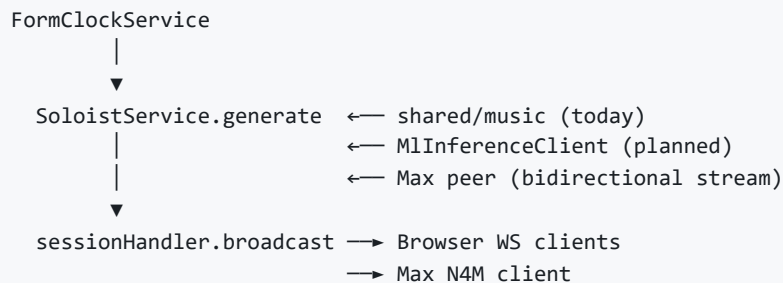
Reuse the typed messages already defined in `shared/types`. Max speaks the same vocabulary as the browser jam client.

DIRECTION	MESSAGE	MEANING ON THE STREAM
Max → App	<code>midi_input</code>	Live notes into the player buffer (gated today to <code>player_solo</code> phase)
App → Max	<code>session_state</code>	Phase, chorus, BPM, form progress — Max's sync source
App → Max	<code>soloist_midi_out</code>	Generated line for this chorus (batch of beat-timed notes)
App → Max	<code>backing_track_ready</code>	Bass / drums / comp parts Max can schedule or ignore
App → Max	<code>solo_analysis_ready</code>	Theory analysis payload; Max may ignore or visualize

Timing property. Generation today is chorus-boundary *batch*, not note-by-note model tokens. The “stream” is a live control + MIDI event channel with occasional note-list payloads—not a continuous audio pipe. Max schedules from `startBeat / durationBeats` plus BPM from `session_state`.

5. Where the stream sits in the stack

INSERTION POINT



Default construction: Max connects to the same `/ws/sessions/:id` path. Optionally tag with `?role=max` later if you need asymmetric filtering. A dedicated `/ws/max/:id` path is only warranted when Max and browser message needs diverge.

6. How session features work through Max

The jam features already in Jazz Gen do not need reinventing for Max. They ride the same messages; Max becomes another listener and/or another input surface.

Session setup & start

Chart, BPM, trade length, who solos first, and band feel are still configured in the browser. Max does not own session creation in v1. After `POST .../start`, both clients receive `session_state` and (usually) `backing_track_ready`. Max loads the session id into the bridge and locks to that jam.

Form clock & turns

`FormClockService` advances beats and flips player ↔ soloist turns at chorus boundaries. Those updates arrive as `session_state`. In Max this can drive transport slaves, phase lights, clip triggers, or gates (“only listen during player solo”). Max should *follow* this clock, not run a competing master transport.

Player MIDI input

Today the browser sends Web MIDI as `midi_input` during `player_solo`. With Max connected, a keyboard or controller patched into Max can send the same message shape through N4M. Notes land in the same player buffer the soloist trade engine already reads. Dual input (browser + Max) is possible; both write into one session.

Soloist generation & motif trade

At a soloist turn, `SoloistService` either builds an autonomous line or answers the player’s motif (if enough notes were captured). The result is broadcast as `soloist_midi_out`—a list of beat-timed notes for that chorus. The browser schedules them on SF2; Max can schedule the same list on its own synths, external MIDI gear, or further processing. One generation event, two (or more) renderers.

Backing band

`backing_track_ready` carries bass, drums, and comp parts as MIDI note lists (plus optional energy: chill / steady / push). The browser loops them in FluidSynth. Max can ignore them, play a subset (e.g. drums only), or replace the browser band entirely with higher-quality Max instruments while still taking cues from energy updates after turn flips.

Band energy

After certain chorus flips the backend may refresh backing energy. That rides on the backing payload. Max patches can map energy to dynamics, density, or FX sends—keeping the “band reacts to the solo” feel outside the browser.

Stop & analysis

Stopping in the browser stops the clock and produces `solo_analysis_ready`. Max typically goes idle or disconnects. Optionally, Max can visualize analysis (scale-fit heat, phrase boundaries) for a studio or classroom patch—without owning the analysis engine.

User mental model: one jam, two surfaces (web stage + Max patch), one shared beat. Features stay owned by Jazz Gen; Max is how those features enter a deeper audio / MIDI / performance environment.

7. Use cases Max can serve

Max is valuable beside Jazz Gen because it is already the place many musicians build custom instruments, route hardware, and design interactive systems. These use cases assume the bidirectional peer stream above.

RENDERER

High-end / custom soloist voice

Jazz Gen generates the line; Max plays it through bespoke synths, sampled horns, granular processing, or outboard MIDI modules. The browser can keep a preview SF2 while Max is the performance voice for recording or stage.

INPUT SURFACE

Studio controller & hardware MIDI

Controllers, wind MIDI, pedals, and exotic interfaces that are awkward in the browser attach cleanly in Max. Notes and control still arrive as `midi_input`, so trading and analysis behave as they do today.

BAND REPLACEMENT

Max as the house band

Use `backing_track_ready` as a score: Max's bass, drums, and comp replace or augment browser SF2. Energy levels from the session drive arrangement intensity. Useful when the web band is "good enough to practice" but Max is "good enough to record."

PERFORMANCE SYSTEM

Live visuals & show control

`session_state` (phase, chorus, BPM) and analysis events become cues for lighting, video, clip launchers, or generative visuals timed to form—without Max needing to understand jazz theory itself.

PEDAGOGY

Teaching & practice lab

A classroom patch mirrors turn badges, highlights active chords, or freezes the last soloist phrase for slow playback. Students jam in the browser; the instructor's Max patch is the shared "board."

RESEARCH / ML HOST

External model or Max-resident generator

When an ML soloist (or a Max-native generative patch) is ready, Max can produce note lists in the same shape as `soloist_midi_out`. Jazz Gen remains the orchestrator and browser mirror; Max becomes the generation peer. The stream layer does not change—only the producer behind the message does.

HYBRID DAW BRIDGE

Route into Ableton, hardware, or multi-outs

Max sits between Jazz Gen and a larger studio graph: channelize soloist vs. band, clock other software via Max's MIDI/clock outs, or record stems while the web UI stays the session control surface.

8. Max-side construction

Inside the patch, the bridge is deliberately thin:

```
[node.script jazzgen-bridge.js]
| outlets → [midiout] / [noten] / dict routing / UI
| inlets  ← [midiin] / bangs / session-id field
```

N4M script responsibilities

1. Open a WebSocket to the backend with the session id.
2. On message: `JSON.parse` → `Max.outlet` or `Max.setDict` (note lists, clock fields).
3. On Max inlet (`Max.addHandler`): pack `midi_input` / control → `ws.send(JSON.stringify(...))`.
4. Reconnect with backoff on close; expose connected / disconnected in the patch UI.

Beat translation: convert `startBeat` + session BPM into milliseconds or Max transport time. Prefer following Jazz Gen's clock from `session_state` so the browser and Max do not drift.

9. Latency & constraints

- **Payload is MIDI events, not audio.** Localhost control latency is usually fine; musical tightness depends on a shared beat origin and ahead-of-time scheduling of note batches.
- **soloist_midi_out is a batch.** Max should schedule the phrase (like the browser's MIDI playback), not treat packets as live note-on streams—unless generation itself is later changed to stream note-by-note.
- **Phase gate.** `midi_input` is ignored unless `phase === 'player_solo'`; Max inherits that rule.
- **Sessions are in-memory and anonymous.** Fine for local Max; not for internet Max without auth later.
- **CORS** only affects browser HTTP. Raw WS from N4M to localhost is unaffected.

10. Build phases

- 1 **Proof of pipe** — N4M connects to `/ws/sessions/:id` and prints `session_state` in the Max console.
- 2 **App → Max MIDI** — Map `soloist_midi_out` to Max MIDI outlets; play a simple synth.
- 3 **Max → App MIDI** — Map Max `midiin` to `midi_input`; confirm notes land in the player buffer.
- 4 **Clock follow** — Slave Max metro/transport to BPM + phase from `session_state`.
- 5 **Optional split** — `role=max` filtering or `/ws/max/:id` only if needed.
- 6 **Later** — If Max/ML becomes the generator, swap the producer behind `SoloistService`; keep message shapes.

11. Non-goals (for this architecture)

- Streaming WAV/PCM over WebSocket (the stack is MIDI-event based).
- Replacing OSC inside Max (OSC may still be used inside the patch; the app↔Max link is WS JSON for parity with the browser).
- Making Max the form-clock master in v1 (that would fight `FormClockService`).